# AI4Science Course
## Class 5 - More Unsupervised Learning

Gabriella (Gaby–the other one) Contardo

# Previously in AI4Science course…

- Linear Regression
- Classification
- Perceptron and Deep Neural Networks: multi-layer perceptron, CNN, ResNet, UNet
- Gradient descent and backpropagation
- Principal Component Analysis
- Preprocessing tips and tricks, classical machine learning setup and evaluation
- Introduction to unsupervised learning:
  - Clustering
  - Anomaly detection
  - Dimensionality Reduction

# This course

Please feel free to interrupt if you have questions or if you're lost!!

- What are some of the problems and tasks that we can tackle "unsupervised" with neural networks? Why and how? What are the advantages of neural nets, and the caveats?

→ A variety of problems/objectives under that umbrella

**Goals**:

- Overview of three different regimes with key methods:

– Dimensionality Reduction – Generative Methods – Density Estimation

- Intuitions about the ideas behind the models, some concepts and limitations

# Outline

- Questions about the previous lectures?
- Dimensionality Reduction with deep neural networks
  - Auto-encoders
  - A bit of history & Concepts
- Generative AI
  - Variational Autoencoder
  - GANs
  - Diffusion models
- Density Estimation
  - ~~Kernel Density Estimation~~
  - Normalizing Flows

# Some ressources

- https://www.deeplearningbook.org/
- https://lilianweng.github.io/posts/2018-08-12-vae/ (and other blog posts e.g. GANs, Diffusion, Normalizing Flows)
- https://huggingface.co/learn/computer-vision-course/en/unit5/generative-models/variational_autoencoders
- https://harryknee.github.io/posts/2024/vae/
- https://pureai.com/articles/2020/05/07/variational-autoencoders.aspx
- https://towardsdatascience.com/why-do-we-minimize-the-mean-squared-error-3b97391f54c/ and https://rish-01.github.io/blog/posts/ml_estimation/ for MSE <-> MLE
- GANs : https://www.geeksforgeeks.org/deep-learning/generative-adversarial-network-gan/
- Diffusion : https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models
- https://www.youtube.com/watch?v=iv-5mZ_9CPY&t=647s
- Normalizing flows: https://deepgenerativemodels.github.io/notes/flow/

Other stuff:

- https://playground.tensorflow.org/
- t-SNE: https://distill.pub/2016/misread-tsne/

# Dimensionality Reduction

- Methods in the previous classes for dimensionality reduction: PCA, UMAP, feature selection
- Different motivations behind dimensionality reduction:
    - Removing correlated or “redundant” or “useless” features to potentially improve your models down the line or make them faster
    - Working with large or complex data which original space is not always conducive to use other type of methods
    - Performing data analysis/mining, understanding the potential underlying structure in your dataset (related to above), gain insights on your data
    - Visualization
- Goals and data should drive the choice of method(s) you use
- Neural networks can be used to perform dimensionality reduction!
- Representation learning ⇔dimensionality reduction

# Autoencoders: Intuition

Autoencoder: a type of architecture that aims to *compress* (“encode”) the input into a “latent” representation vector of lower dimension, such that it can reconstruct the original input well. Byproduct: dimensionality reduction

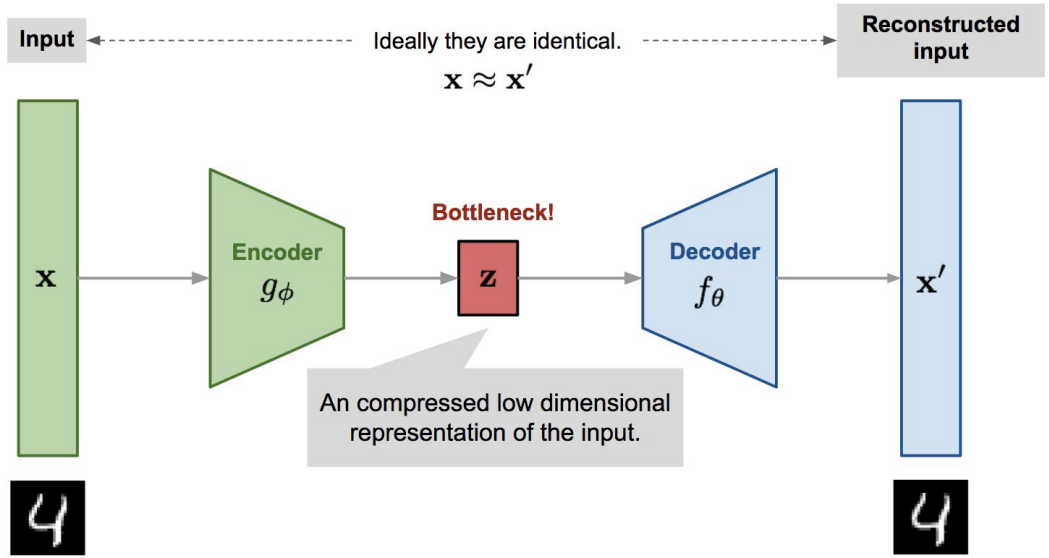



Image credit Lilianweng blog

Encoder: translates the original high-dimension input into the latent low-dimensional vector. input size > output size

Decoder: recovers the data from the encoder output

Train e.g. with a MSE loss between x and x’ (or BCE if binary values)

**“Unsupervised”: the model is supervised by the *data itself acting as its own label*.**

# Autoencoders: Context and history

Autoencoder: a type of architecture that aims to *compress* (“encode”) the input into a “latent” representation vector of lower dimension, such that it can reconstruct the original input well. Byproduct: dimensionality reduction

- Back in the 80s:
  - Oja (1982): 1 layer autoencoder is similar to PCA (requires additional constraints to be actually the same tho)
  - Rhumelhart et al (1986) in the seminal backpropagation paper shows an example of auto-association: point was not so much about compression but about features that were not ‘handcrafted’
  - Baldi & Hornik (1989) Neural Networks & non-linear PCA
  - Kramer (1991) : “non-linear PCA” generalization
  - http://www.nlpca.org/

# Autoencoders: Context and history

- Dimensionality Reduction / PCA-like oriented, but also as a procedure to train deeper networks, layer by layer:
  - Hinton & Salakhutdinov (2006) Restricted Boltzman Machine ; Hinton, Osindero, and Teh (2006)
  - Stacked auto-encoders , Bengio et al (2006)

However, until recently, it was believed too difficult to train deep multi-layer neural networks. Empirically, deep networks were generally found to be not better, and often worse, than neural networks with one or two hidden layers (Tesauro, 1992). As this is a negative result, it has not been much reported in the machine learning literature. A reasonable explanation is that gradient-based optimization starting from random initialization may get stuck near poor solutions. An approach that has been explored with

- Why does unsupervised pretraining help deep learning (2010)

## Stacked auto-encoders , Bengio et al (2006)

"However, until recently, it was believed too difficult to train deep multi-layer neural networks. Empirically, deep networks were generally found to be not better, and often worse, than neural networks with one or two hidden layers [..] explanation is that gradient-based optimization starting from random initialization may get stuck near poor solutions. [...] constructively adding layers [..] Hinton, Osindero, and Teh (2006) recently introduced a greedy layer-wise unsupervised learning algorithm [...] Upper layers are supposed to represent more "abstract" concepts that explain the input observation x, whereas lower layers extract "low-level features" from x. **They learn simpler concepts first, and build on them to learn more abstract concepts.** This strategy has not yet been much exploited in machine learning."

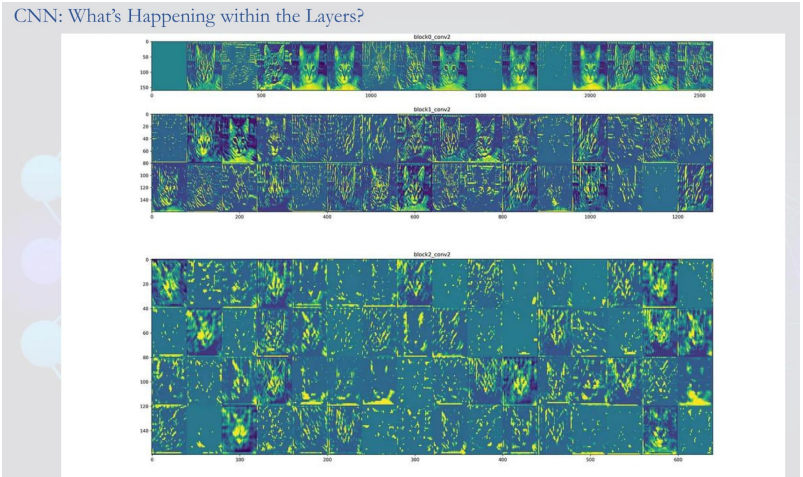
Cf course 2 :

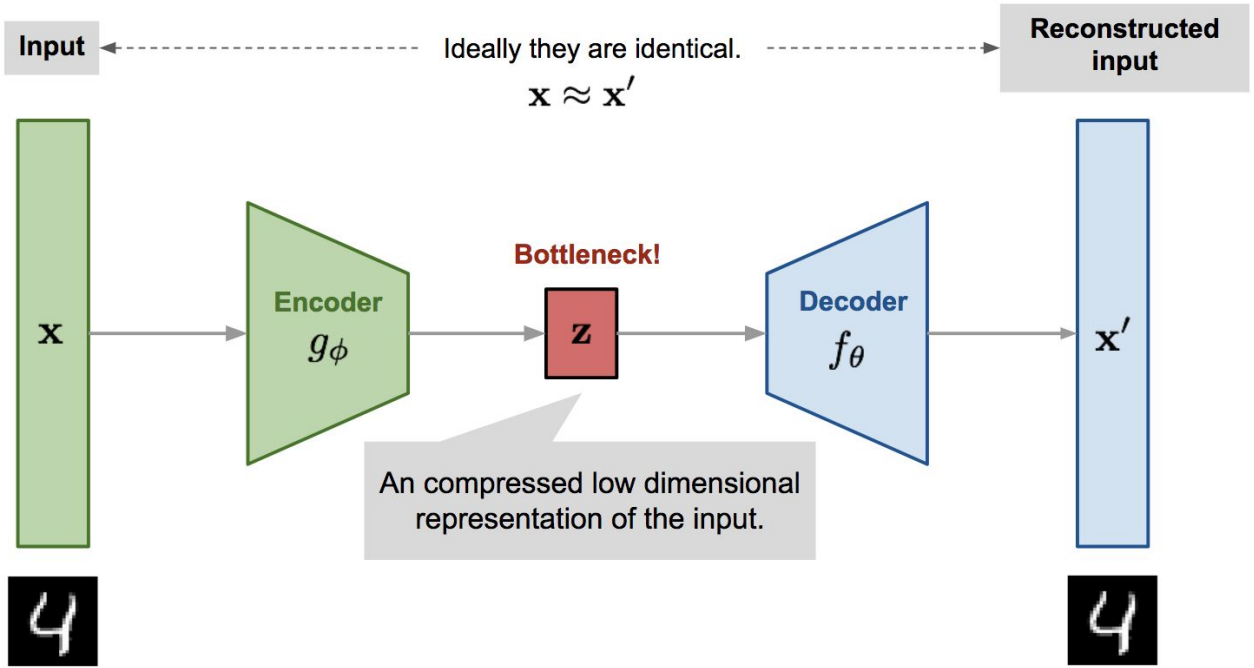
Feature? Layer? ????

Playground tensorflow

## Why does unsupervised pretraining help deep learning (2010)

"Deep learning methods aim at **learning feature hierarchies with features from higher levels of the hierarchy formed by the composition of lower level features**. [...] in order to learn the kind of complicated functions that can represent high-level abstractions, one may need deep architectures. [...] despite the serious challenge of training models with many layers of adaptive parameters. [...] the objective function is a highly non-convex function of the parameters, with the potential for many distinct local minima [...] The breakthrough to effective training strategies for deep architectures [..] layer-wise unsupervised pre-training followed by supervised fine-tuning. Each layer is pretrained with an unsupervised learning algorithm, learning a nonlinear transformation of its input (the output of the previous layer)




Input
Ideally they are identical.
$\mathbf{x} \approx \mathbf{x}'$
Reconstructed input
$\mathbf{x}$
Encoder
$g_\phi$
Bottleneck!
$\mathbf{z}$
Decoder
$f_\theta$
$\mathbf{x}'$
An compressed low dimensional representation of the input.

# Autoencoders

- Modifications to improve robustness
  - Denoising auto-encoders (stacked) (Vincent et al 2008) – 'noise' can be defined in different ways. Force network to extract somewhat more meaningful features (exploiting/detecting correlations) that are robust to noise/corruption. Convenient because also augment data; nice for missing data

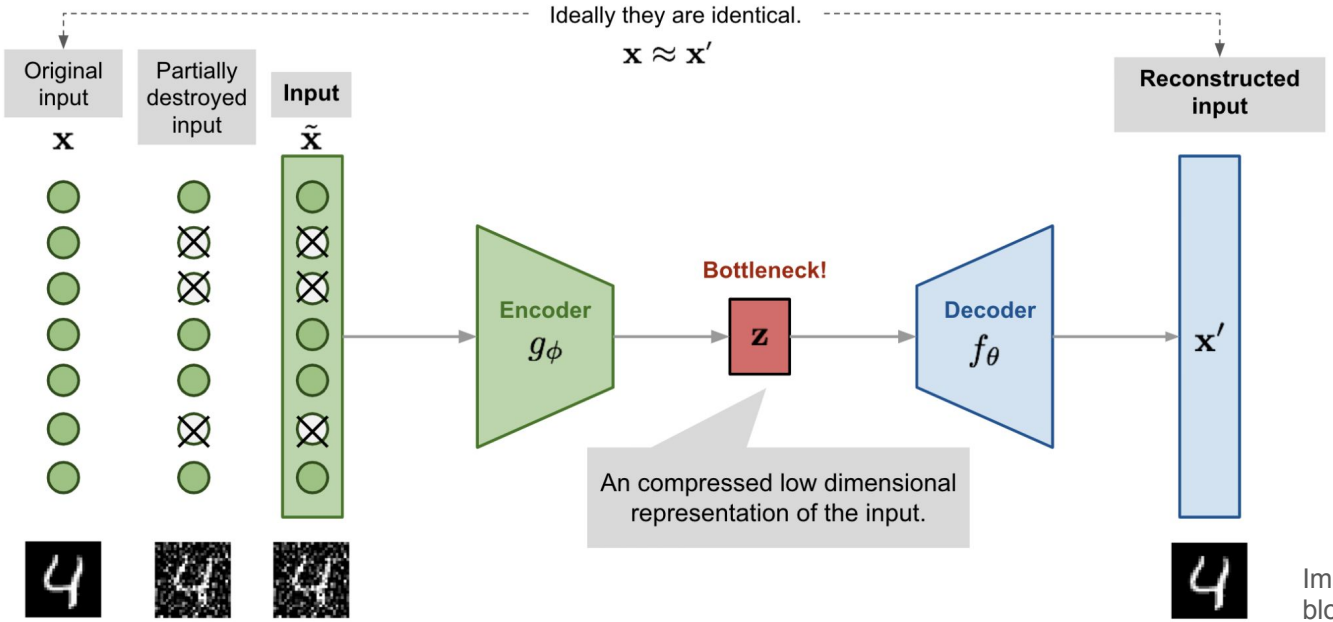
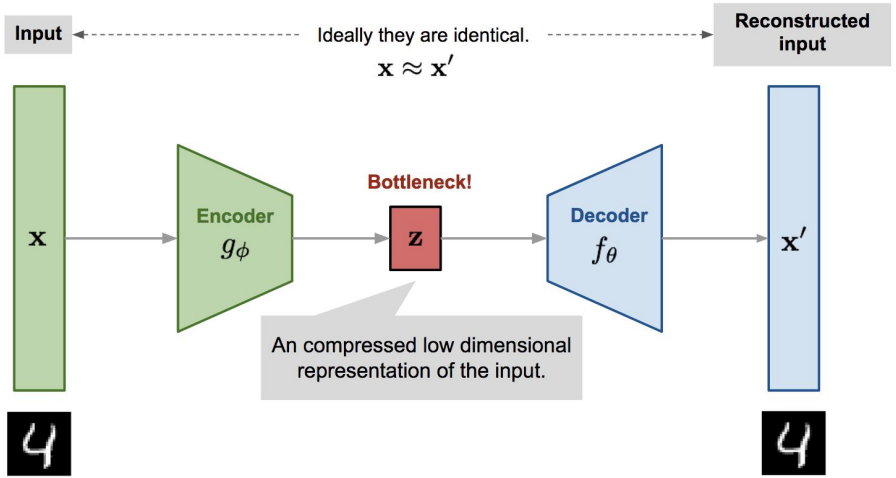



Image credit Lilianweng blog

# Autoencoders: Context and history

- More modifications (90s-2010s)
  - Sparse auto-encoder, k-sparse auto-encoder: impose sparsity constraints on the bottleneck; forces the model to only have a small number of hidden units being activated at the same time

    E.g. in 1996 on a neuro application
  - Contractive auto-encoder: enforce that the bottleneck is not too sensitive to small changes in the input, not by adding noise in the input, but by constructing a second term to the loss using gradient of the bottleneck with respect to the input




| | **PCA** | **Autoencoder** |
|---|---|---|
| Transformation | Linear | Nonlinear (but can be linear) |
| Latent representation (vector) | Ordered by variance (thus: nested); deterministic | Usually unordered (vanilla) |
| Complexity | Low (fast, analytical) | High(er)-ish; stochastic algorithm |
| “Interpretability” | High | Eeeehhhh |

# Some visualizations

[Hinton et al, 2006]

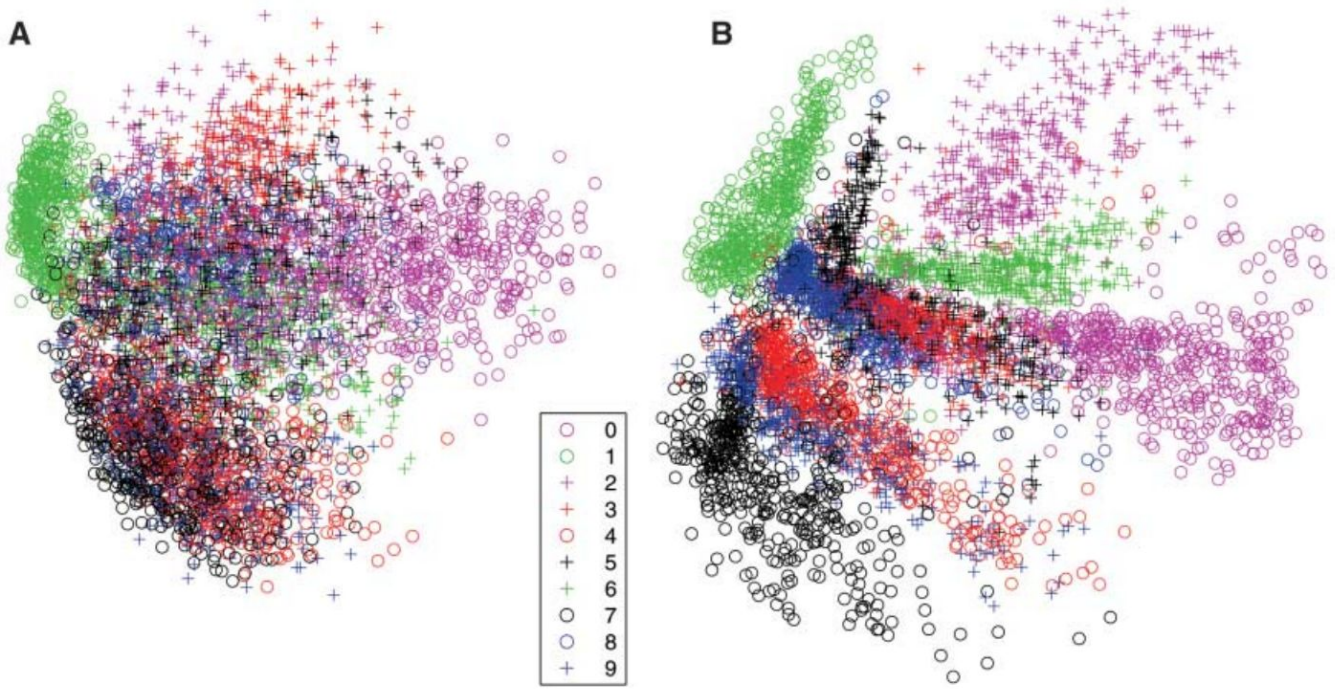
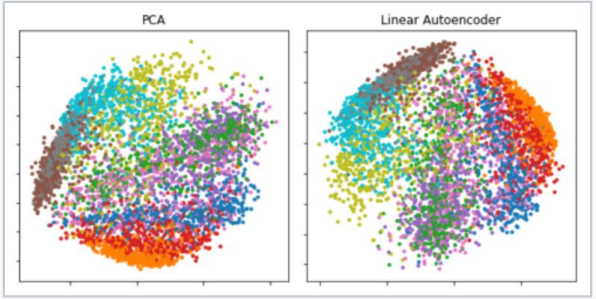



**Fig. 3.** (**A**) The two-dimensional codes for 500 digits of each class produced by taking the first two principal components of all 60,000 training images. (**B**) The two-dimensional codes found by a 784-1000-500-250-2 autoencoder. For an alternative visualization, see (*8*).

# Some visualizations




Plot of the first two Principal Components (left) and a two-dimension hidden layer of a Linear Autoencoder (Right) applied to the Fashion MNIST dataset.[37] The two models being both linear learn to span the same subspace. The projection of the data points is indeed identical, apart from rotation of the subspace. While PCA selects a specific orientation up to reflections in the general case, the cost function of a simple autoencoder is invariant to rotations of the latent space.

Image credit: wikipedia

# Some considerations

- Inductive bias of AE (i.e. the assumption / restriction we make) is that information that can be discarded without hurting reconstruction (hopefully) will be discarded. Forces *compression.*
- This is based on the reconstruction loss, and does not provide any other guarantee of properties on what is going on in your bottleneck
- Relation to manifold learning: assumes that the data *can* be represented by fewer dimensions than the input; however, there's also a relationship with the complexity and expressivity of the encoder/decoder
- Technically, any layer of a deep neural network learns a "representation"!
- Autoencoder <-> concepts from 'dictionnary learning'
- What makes a 'good' representation/compression kinda keeps evolving (e.g. aim for semantic/organization)

Representation Learning: A Review and New Perspectives (2014)

# Some more considerations

- Representation is sufficient for reconstruction: encode everything that is relevant
- *“Everything”* can be a problem: does not privilege any particular factor of variation;
- Examples on MNIST etc can be misleading because the digit class (which we focus on) tend to be the dominant source of variation, and is thus detected; what if we had much more variance in e.g. strock width? Rotations?
- **How would you cluster a deck of cards?**
- If your data has many factors of variations, flying blind through representation learning can be…tricky.

Locatello et al. (2019), Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations

+ Elhage et al (2022); Coenen et al (2019)

# Generative Models / Hybrid

# Variational Autoencoder (VAE) – Stepping into Generative Models

Kingma and Welling, 2014

- Probabilistic framework
- Instead of mapping the input into a fixed vector, we want to map it into a distribution.
- But the architecture/model in practice is very similar to an AE
- Original motivation: variational bayesian inference & probabilistic modeling
- Advantages:
  - We can (potentially) sample from the bottleneck (classical AE have no continuity or distribution guarantee) to create new data
  - We get an evidence lower bound which can be useful

# Variational Autoencoder (VAE) – Stepping into Generative Models

Kingma and Welling, 2014

- Assumes data comes from a random process involving an unobserved continuous random variable z;  (i) a value z is generated from some prior distribution $p_{\theta^*}(z)$, (ii) a value x is generated from some conditional distribution $p_{\theta^*}(x|z)$.
- Assumes that the prior $p_{\theta^*}(z)$ and likelihood $p_{\theta^*}(x|z)$ come from parametric families of distributions $p_\theta(z)$ and $p_\theta(x|z)$, and that their PDFs are differentiable almost everywhere w.r.t. both $\theta$ and z
- A lot of this process is hidden from our view: the true parameters $\theta^*$ as well as the values of the latent variables z are unknown to us.
- Interested specifically in case where the marginal likelihood $p_\theta(x) = \int p_\theta(z) p_\theta(x|z)\, dz$ is intractable

# Variational Autoencoder

VAE proposes a solution to three different scenarios:

1. Efficient approximate ML or MAP estimation for the parameters $\boldsymbol{\theta}$. The parameters can be of interest themselves, e.g. if we are analyzing some natural process. They also allow us to mimic the hidden random process and generate artificial data that resembles the real data.
2. Efficient approximate posterior inference of the latent variable $\mathbf{z}$ given an observed value $\mathbf{x}$ for a choice of parameters $\boldsymbol{\theta}$. This is useful for coding or data representation tasks.
3. Efficient approximate marginal inference of the variable $\mathbf{x}$. This allows us to perform all kinds of inference tasks where a prior over $\mathbf{x}$ is required. Common applications in computer vision include image denoising, inpainting and super-resolution.

# Variational Autoencoder

- Introduce qφ(z|x): an approximation to the intractable true posterior pθ(z|x).

From a coding theory perspective, the unobserved variables $\mathbf{z}$ have an interpretation as a latent representation or *code*. In this paper we will therefore also refer to the recognition model $q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$ as a probabilistic *encoder*, since given a datapoint $\mathbf{x}$ it produces a distribution (e.g. a Gaussian) over the possible values of the code $\mathbf{z}$ from which the datapoint $\mathbf{x}$ could have been generated. In a similar vein we will refer to $p_{\boldsymbol{\theta}}(\mathbf{x}|\mathbf{z})$ as a probabilistic *decoder*, since given a code $\mathbf{z}$ it produces a distribution over the possible corresponding values of $\mathbf{x}$.

- We want the estimate posterior qφ(z|x) to be close to the real one pθ(z|x) (which we don't know but bear with me).
- Kullback-Leibler divergence: quantify the distance between two distributions ($D_{KL}(X|Y)$ measures how much information is lost if Y is used to represent X)
- With a bit of math (check lilian weng's blog for the full thing):

$$D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z}|\mathbf{x})) = \log p_\theta(\mathbf{x}) + D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z})) - \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z})$$

Once rearrange the left and right hand side of the equation,

$$\log p_\theta(\mathbf{x}) - D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z}|\mathbf{x})) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) - D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x})\|p_\theta(\mathbf{z}))$$

The LHS of the equation is exactly what we want to maximize when learning the true distributions: we want to maximize the (log-)likelihood of generating real data (that is $\log p_\theta(\mathbf{x})$) and also minimize the difference between the real and estimated posterior distributions (the term $D_{\mathrm{KL}}$ works like a regularizer). Note that $p_\theta(\mathbf{x})$ is fixed with respect to $q_\phi$.

The negation of the above defines our loss function:

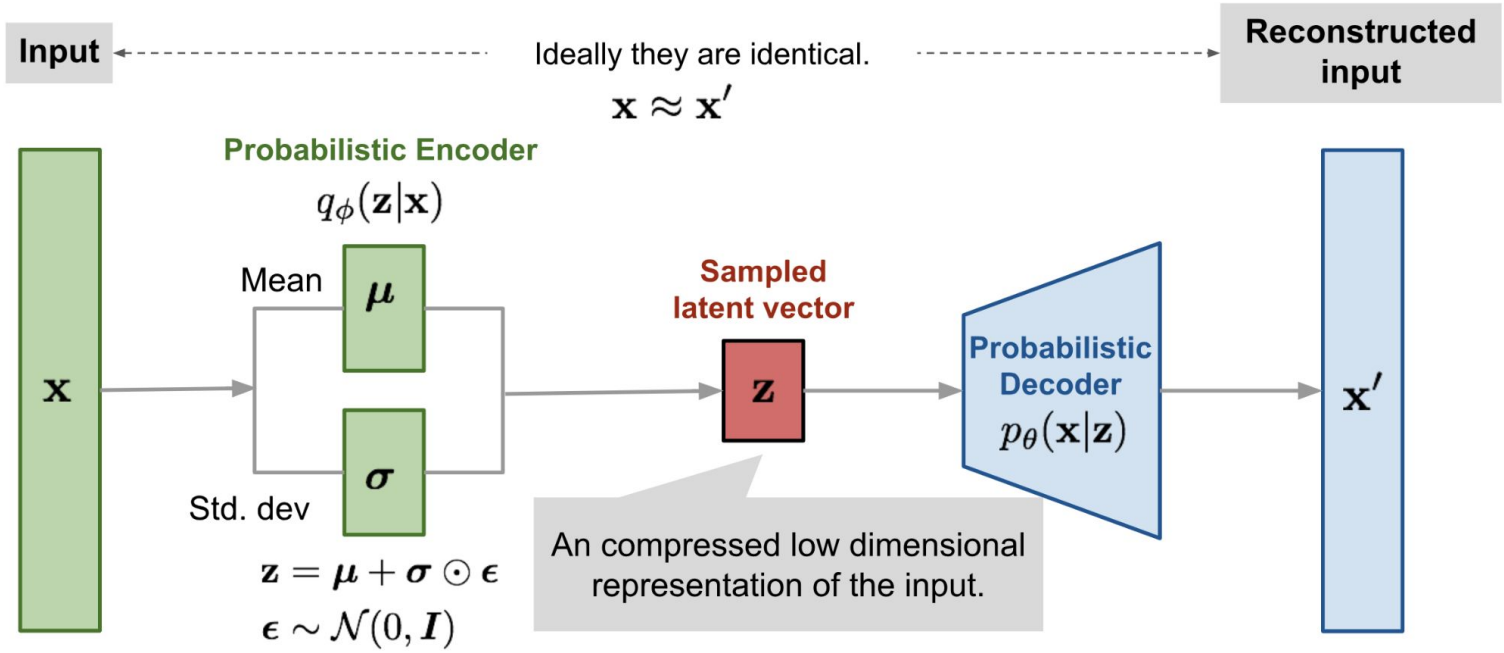
$$
\begin{aligned}
L_{\text{VAE}}(\theta, \phi) &= -\log p_\theta(\mathbf{x}) + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}|\mathbf{x})) \\
&= -\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \log p_\theta(\mathbf{x}|\mathbf{z}) + D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z})) \\
\theta^*, \phi^* &= \arg\min_{\theta,\phi} L_{\text{VAE}}
\end{aligned}
$$

In Variational Bayesian methods, this loss function is known as the *variational lower bound*, or *evidence lower bound*. The "lower bound" part in the name comes from the fact that KL divergence is always non-negative and thus $-L_{\text{VAE}}$ is the lower bound of $\log p_\theta(\mathbf{x})$.

$$
-L_{\text{VAE}} = \log p_\theta(\mathbf{x}) - D_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \| p_\theta(\mathbf{z}|\mathbf{x})) \leq \log p_\theta(\mathbf{x})
$$

- We can evaluate the KL term according to pθ(z) (the prior) because we can choose the prior (usually say a Gaussian)
- This, under the assumptions of the output of the decoder being a gaussian, is a MSE
- Which makes sense: we want z to match our prior, and our x to look ok.

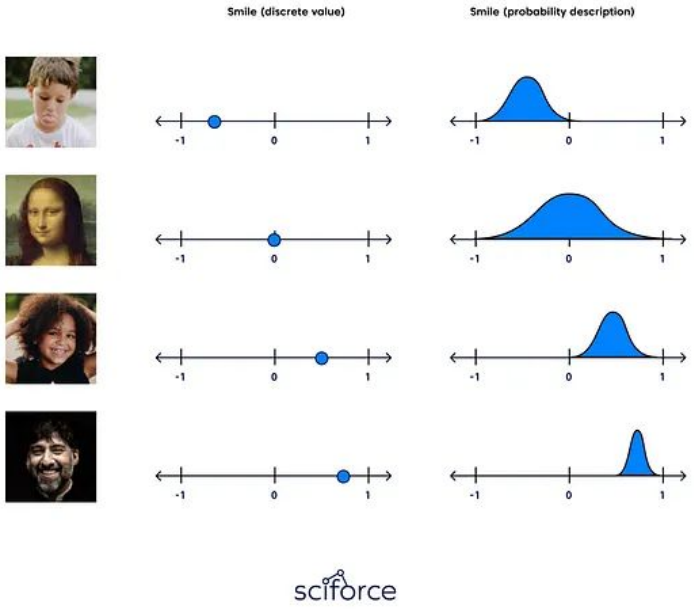
Image credit Lilianweng blog




- Loss optimizes: z looks like the prior we want, x' is well reconstructed according to x.

# Beta VAE

- Disentangled representation: each variable in z is sensitive to one single generative factor / factor of variation;
- Interpretability (ish), good for downstream tasks
- Beta-VAE (Higgins et al (2017)): emphasizes to discover “disentangled representations”
- Namely: adds a β factor to the KL part of the loss; when β>1 : stronger constraints to the bottleneck
- See also Burgess et al (2017)



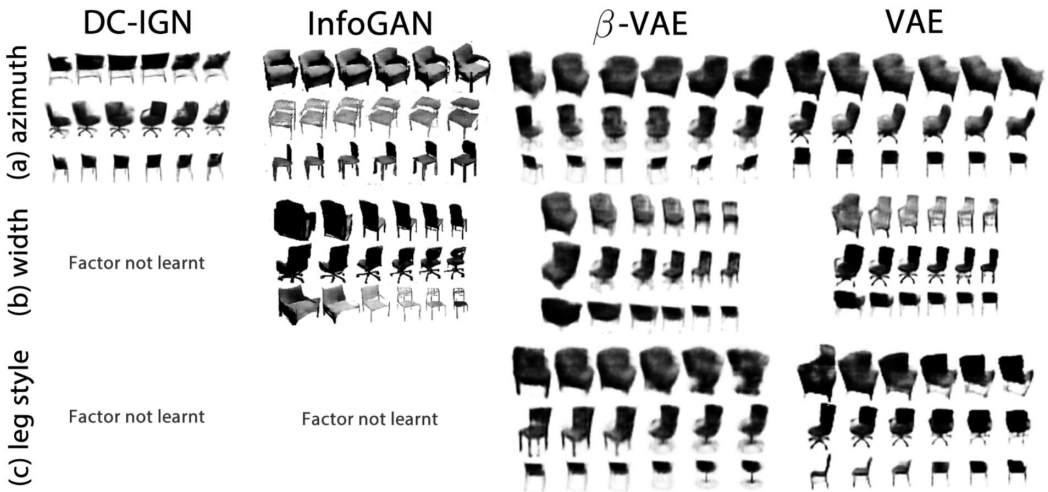




Figure 2: **Manipulating latent variables on 3D chairs:** Qualitative results comparing disentangling performance of $\beta$-VAE ($\beta = 5$), VAE (Kingma & Welling, 2014) ($\beta = 1$), InfoGAN (Chen et al., 2016) and DC-IGN (Kulkarni et al., 2015). InfoGAN traversal is over the [-1, 1] range. VAE always

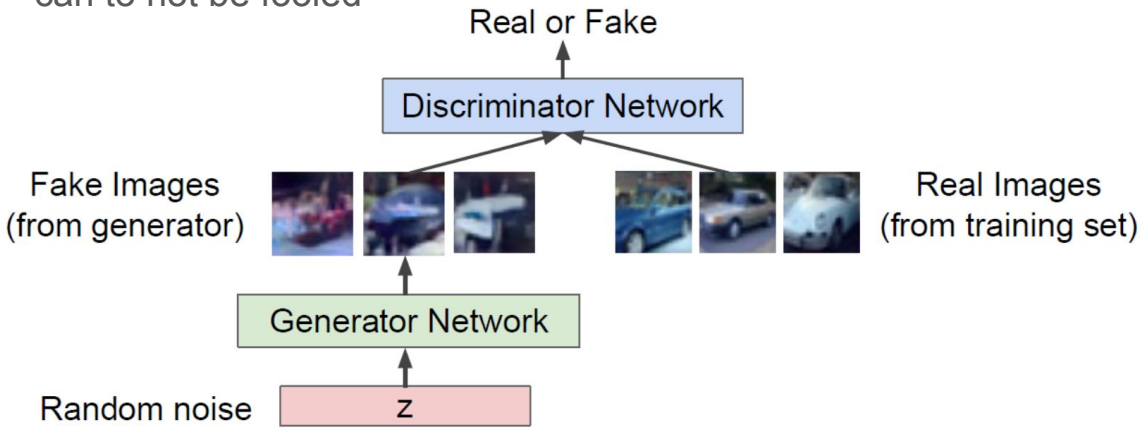
# Notebook AE & VAE

https://tinyurl.com/aevaeung

# More generative models?

# Generative Adversarial Networks (GANs)

- Use the idea of “adversarial” training (which is pretty cool and fun)
- Basically a minimax game between two models that “play” against each other:
  - Model A generates images (**Generator**) from random noise
  - Model B votes if an image is real or fake (**Discriminator**)
  - Generator tries to fool Discriminator as much as it can, Discriminator tries to be as good as it can to not be fooled




$$min_G\ max_D(G, D) = [\mathbb{E}_{x \sim p_{data}}[log\ D(x)] + \mathbb{E}_{z \sim p_z(z)}[log(1 - D(g(z)))]$$

# GANs

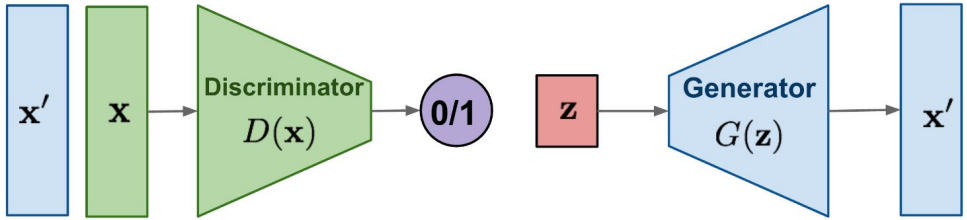
- Very fun idea
- Massive pain to train
- Prone to mode collapse: generated output not as diverse as original distribution, losing modes of distribution
- Hard to evaluate
- Some tricks were proposed (see e.g this or that)
- Extension to e.g. Wasserstein GANs, which try to minimize the Wasserstein distance (or an estimation of it) between generated and true samples: good to improve on mode collapse

# Diffusion Models

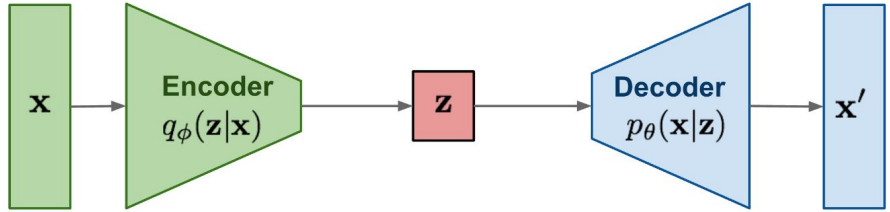
- Another family of generative models
- With subfamilies: diffusion probabilistic model Sohl-Dickstein et al (2015) ; score-based Song et al (2020) ; denoising diffusion probabilistic model Ho et al (2020); …
- General idea is to slowly (iteratively) destroy the structure in a data distribution through a '*forward diffusion process*', and then *learn* a reverse diffusion process that restores the structure.

The sweet child of GANs and denoising AE trick?  (no compression)

**GAN:** Adversarial training




**VAE:** maximize variational lower bound




**Diffusion models:** Gradually add Gaussian noise and then reverse

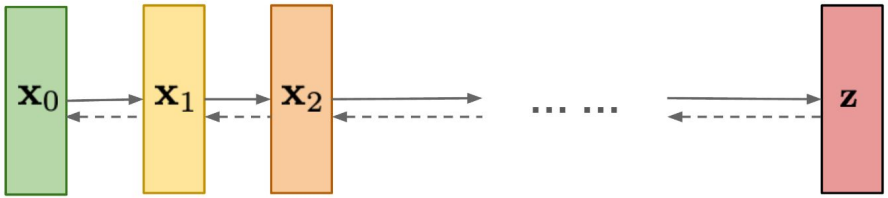
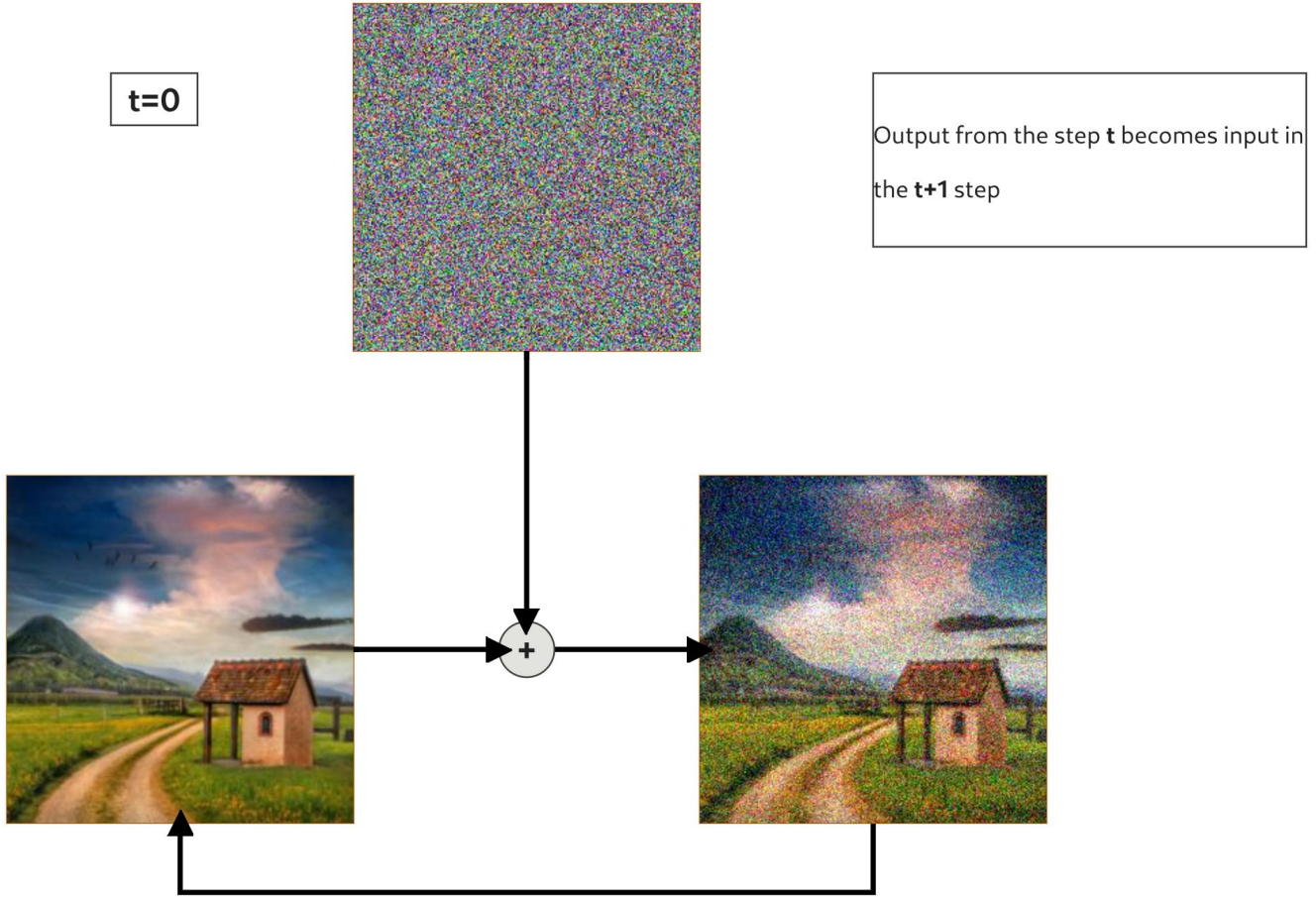
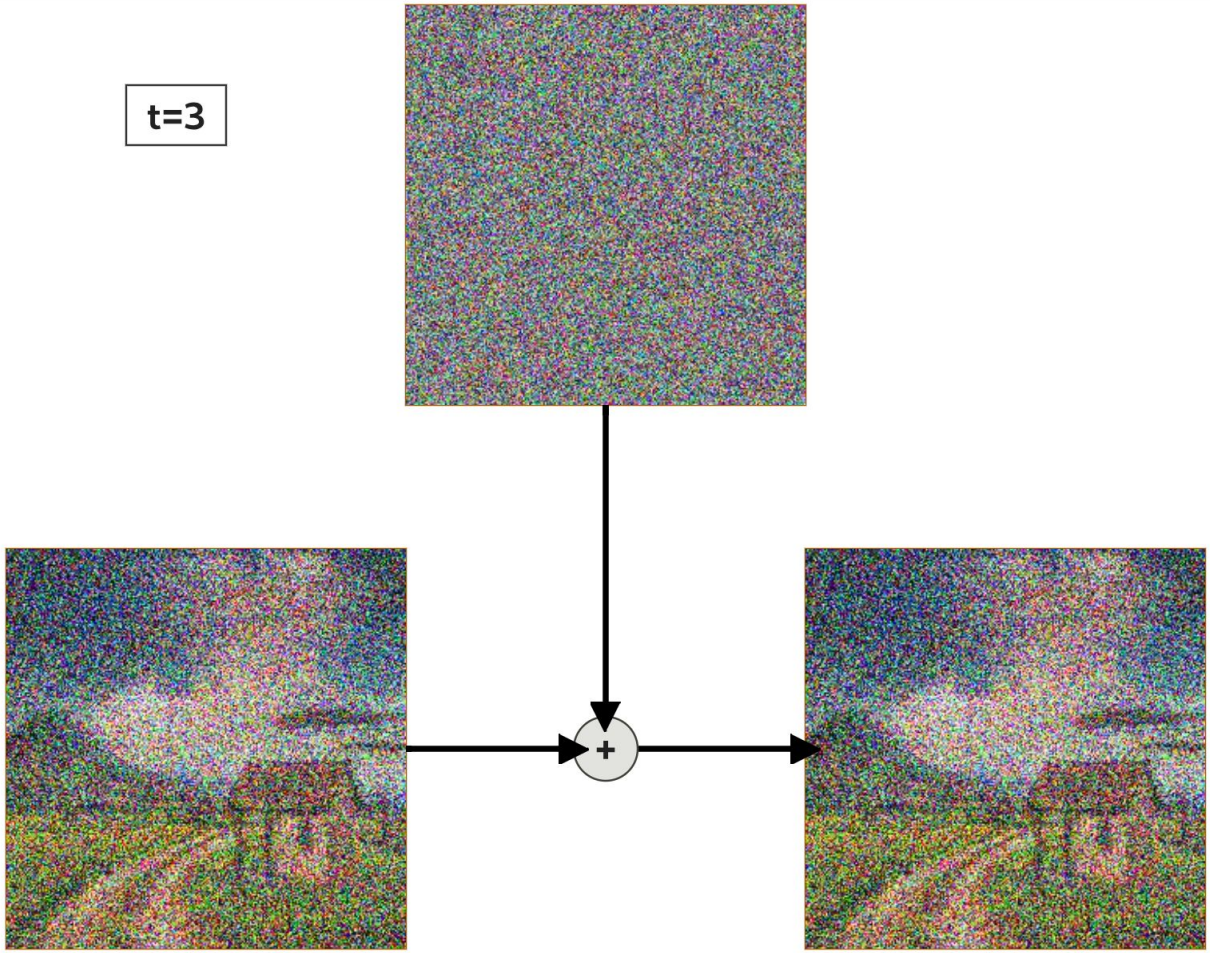



Image credit Lilianweng blog

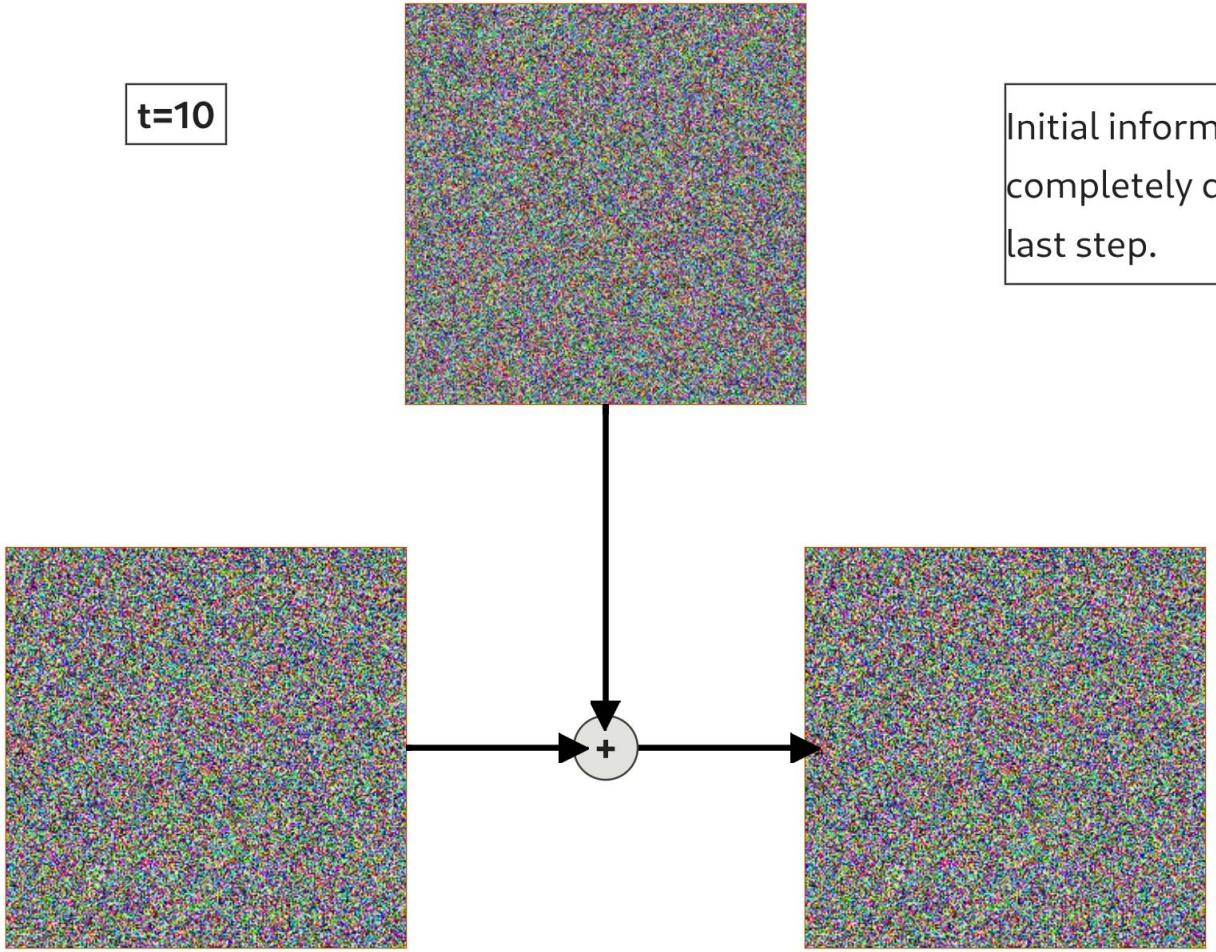



Denoising Diffusion Probabilistic Models – DDPM

https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models


t=3
+

t=10



Initial information is completely destroyed at the last step.

Each step of the forward diffusion process is defined as $q(x_t|x_{t-1}) = \mathcal{N}(x_t, \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$. Where **q** is our forward process, $x_t$ is the output of the forward process at step **t** (naturally $x_{t-1}$ is an input at step **t**). **N** is a *normal distribution*, $\sqrt{1-\beta_t}x_{t-1}$ is our *mean* and finally $\beta_t I$ defines a *variance*.


*Figure 5: Forward diffusion process using **cosine schedule**.*

$\beta_t$ refers to the **schedule** ; values can range from 0 to 1. The values are usually kept low to prevent variance from exploding

The training process doesn't use examples in line with the forward process but rather it uses samples from arbitrary timestep t.

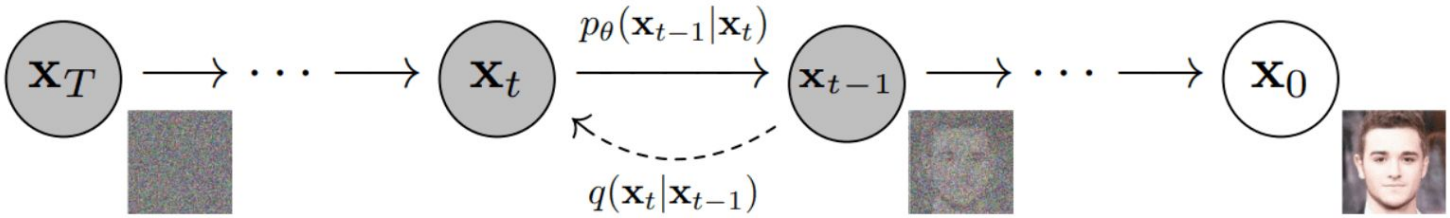
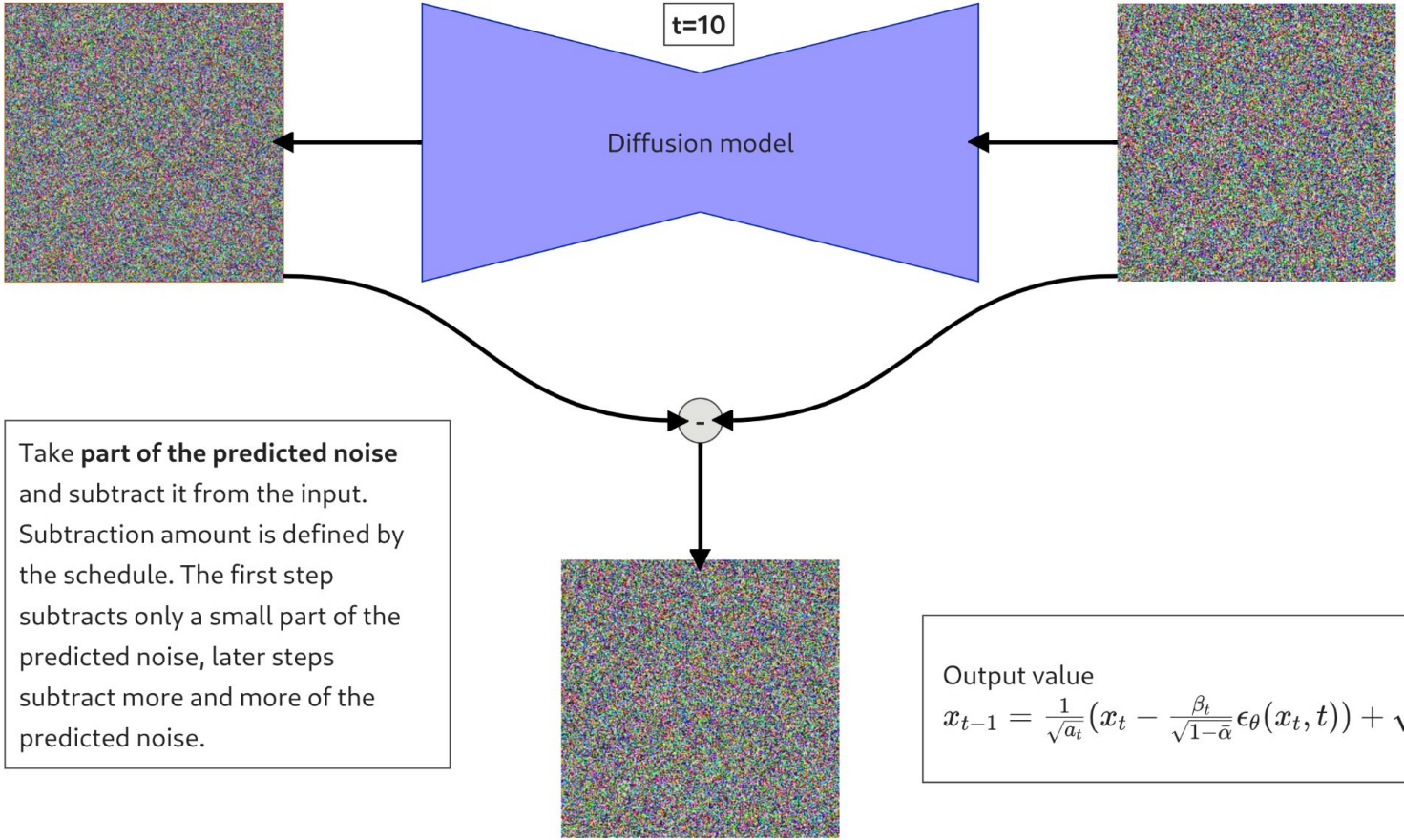



*Figure 7: The directed graphical model. (source: [Deep Unsupervised Learning using Nonequilibrium Thermodynamics, arXiv:1503.03585, 2015.](#))*

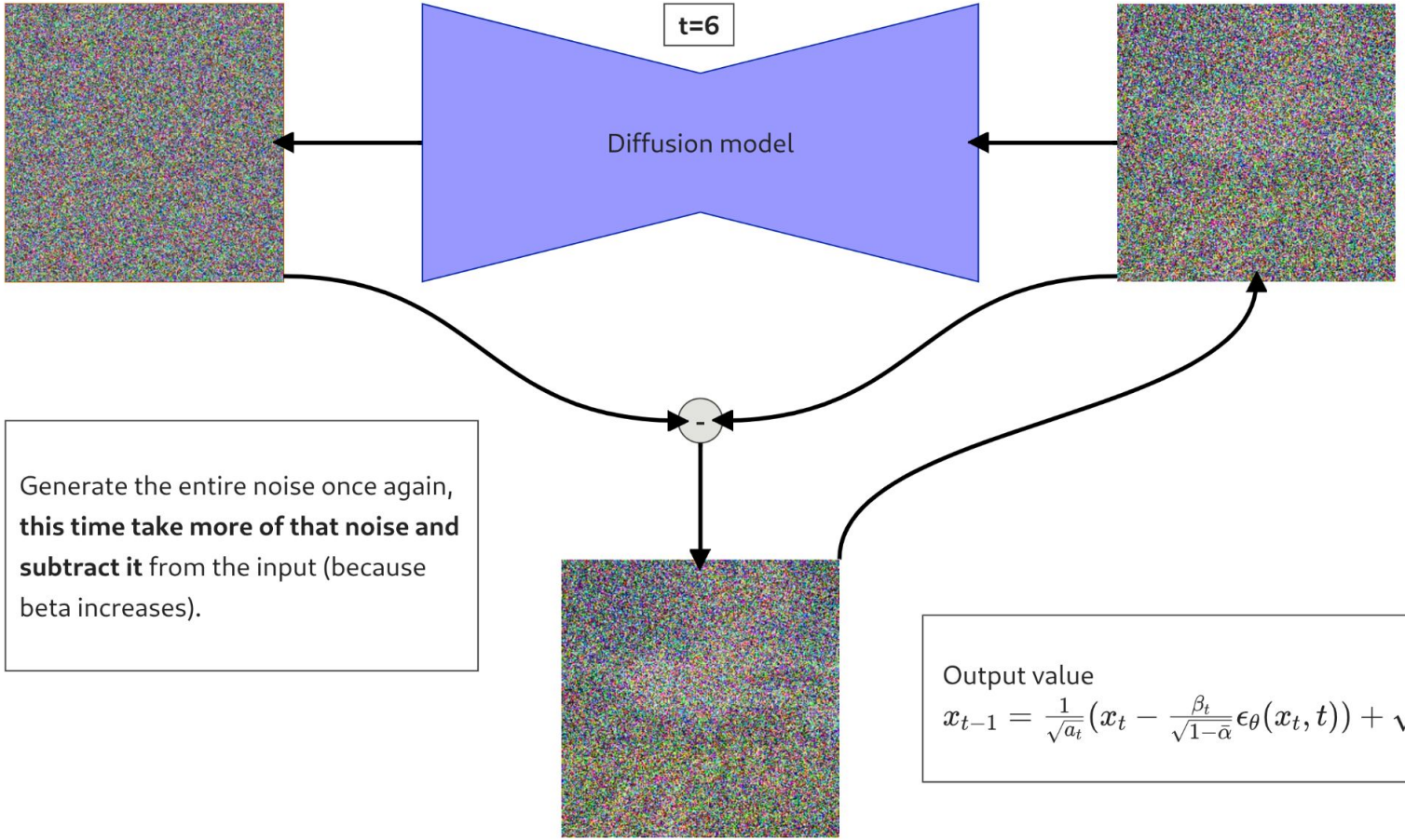
**Diffusion model predicts the entire noise to be removed in a given timestep:** if we have timestep t=600 then our Diffusion model tries to predict the entire noise to remove to get to t=0 (not t=599).




Take **part of the predicted noise** and subtract it from the input. Subtraction amount is defined by the schedule. The first step subtracts only a small part of the predicted noise, later steps subtract more and more of the predicted noise.

Output value

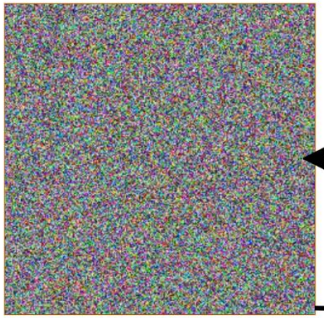
$$x_{t-1} = \frac{1}{\sqrt{a_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}}}\epsilon_\theta(x_t, t)) + \sqrt{\beta_t}\epsilon$$




Generate the entire noise once again, **this time take more of that noise and subtract it** from the input (because beta increases).

Output value

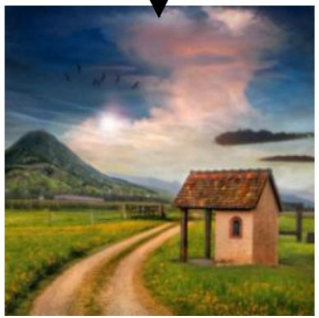
$$x_{t-1} = \frac{1}{\sqrt{a_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}}}\epsilon_\theta(x_t, t)) + \sqrt{\beta_t}\epsilon$$



t=1

Diffusion model

-



Generate entire noise once again, **this time take more of that noise and subtract it** from the input (because beta increases).

**Notice!**

Last step doesn't add noise at the end of the forumla ($\sqrt{\beta_t}\epsilon$)

Output value

$$x_{t-1} = \frac{1}{\sqrt{a_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}}}\epsilon_\theta(x_t, t))$$

**Algorithm 1** Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

**Algorithm 2** Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Many credits to https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models

More maths? Check lilian weng's blog (or read the papers, of course)

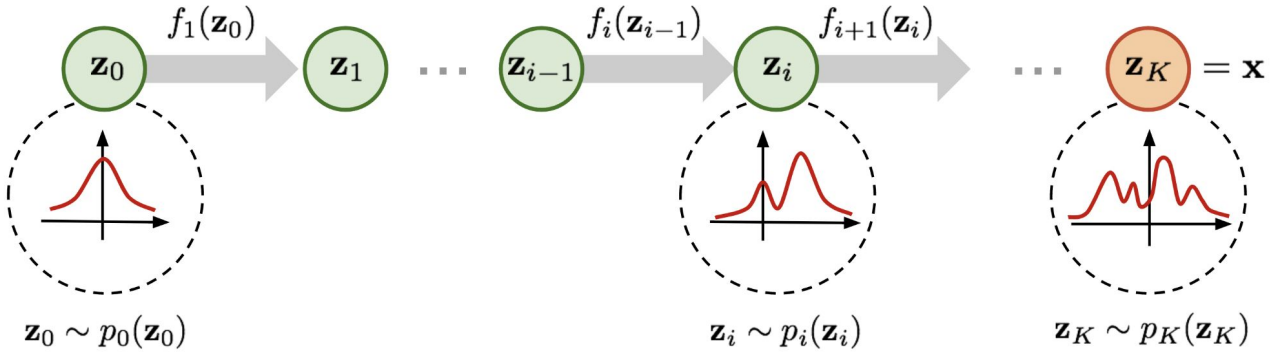
# Density Estimation

# Density Estimation with Neural Nets

- Dimensionality reduction: gives you a smaller size representation of your data
- Generative models like VAEs, GANs, Diffusion models: we can sample (generate new data), get a lower bound (for some); not necessarily dimensionality-reduction.
- These however don't allow to actually estimate or model p(x) i.e. a probability (or density) estimate on your data
- This is what many people want, a lot of the time.
- It's not easy.
- Please look at Kernel Density Estimation since we don't cover it here. And think of the curse of dimensionality.

# Normalizing Flows

- Mapping from a simple distribution (easy to sample and evaluate densities on) to more complex ones (from the data)



# Normalizing Flows

**Change of variables formula**: how to evaluate densities of a random variable that is a deterministic transformation from another variable.

**Change of Variables**: $Z$ and $X$ be random variables which are related by a mapping $f : \mathbb{R}^n \to \mathbb{R}^n$ such that $X = f(Z)$ and $Z = f^{-1}(X)$. Then

$$p_X(\mathrm{x}) = p_Z(f^{-1}(\mathrm{x})) \left| \det \left( \frac{\partial f^{-1}(\mathrm{x})}{\partial \mathrm{x}} \right) \right|$$

1. x and z need to be continuous and have the same dimension.
2. $\frac{\partial f^{-1}(\mathrm{x})}{\partial \mathrm{x}}$ is a matrix of dimension $n \times n$, where each entry at location $(i, j)$ is defined as $\frac{\partial f^{-1}(\mathrm{x})_i}{\partial x_j}$. This matrix is also known as the Jacobian matrix.
3. $\det(A)$ denotes the determinant of a square matrix $A$.

https://deepgenerativemodels.github.io/notes/flow/

# Normalizing Flows

Normalizing flow: mapping between Z and X (data) given by a function fθ, deterministic and invertible such that

$$X = f_\theta(Z) \text{ and } Z = f_\theta^{-1}(X)$$

Using change of variables, the marginal likelihood $p(x)$ is given by

$$p_X(\mathrm{x};\theta) = p_Z(f_\theta^{-1}(\mathrm{x}))\left|\det\left(\frac{\partial f_\theta^{-1}(\mathrm{x})}{\partial \mathrm{x}}\right)\right|$$

# Normalizing Flows

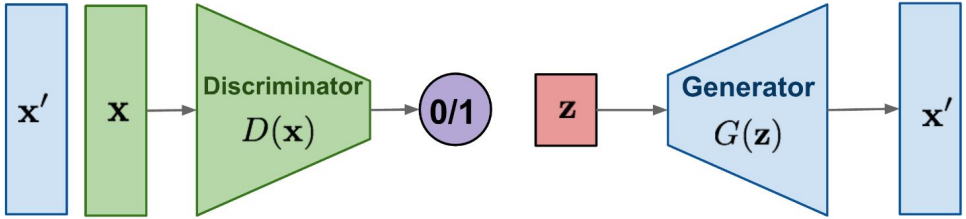
1. “Normalizing” means that the change of variables gives a normalized density after applying an invertible transformation.
2. “Flow” means that the invertible transformations can be composed with each other to create more complex invertible transformations.

Deep normalizing flow models require specific architectural structures.

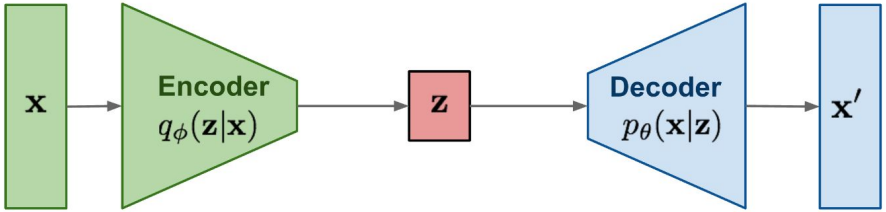
1. The input and output dimensions must be the same.
2. The transformation must be invertible.
3. Computing the determinant of the Jacobian needs to be efficient (and differentiable).

Different architectures/”flows” have been proposed: Nonlinear Independent Components Estimation (NICE) model and Real Non-Volume Preserving (RealNVP), GLOW, Masked Autoregressive Flows (MAF),...

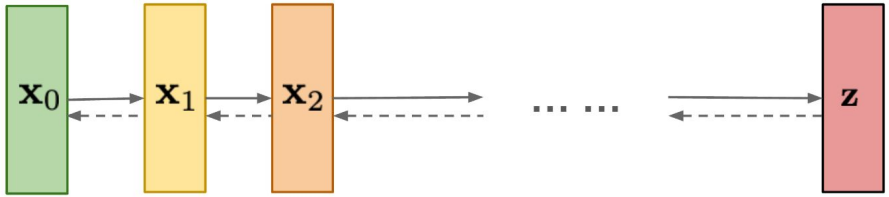
**GAN:** Adversarial training



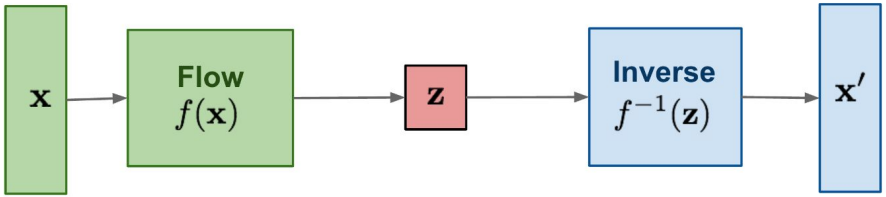

**VAE:** maximize variational lower bound




**Diffusion models:** Gradually add Gaussian noise and then reverse




**Flow-based models:** Invertible transform of distributions



# Normalizing flows

- Notebook : https://tinyurl.com/normflowung

Considerations:

- Out of distribution detection: somehow, normalizing flows still aren't so good at detecting data that are 'out of distribution' https://arxiv.org/abs/2006.08545
- High-dimensionality / lower-manifold interplay: hard to completely evaluate
- Use with caution, but I'm not aware of anything better for high-dimensional data anyway.

# Wrap-up

- **AEs and VAE**s: encoder-decoder, latent space is lower-dimensional: explicit compression, but through reconstruction (limitations/bias); VAEs has additional properties that can influence/constraints the representation space+ELBO
- **GANs**: latent (random) space -> data space only, no encoder, latent dimension is a design choice (often lower than data, but doesn't have to be), not “reconstruction”
- **Diffusion models**: iterative, also same dimension, also no compression, to generate from ‘noise’
- **NFs**: bijective mapping, latent space is the same dimension as the data, no compression, just a change of variables; can sample but can also estimate p(x) (...to some extent)
- For semantic/structure/similarity constraints: **contrastive methods** (SimCLR, CLIP) and **masked autoencoders** (MAE)
- Keep in mind: Inductive bias + compression + deck of cards

Congrats, it’s friday 4pm!


Image credit: me

# Inside baseball slides

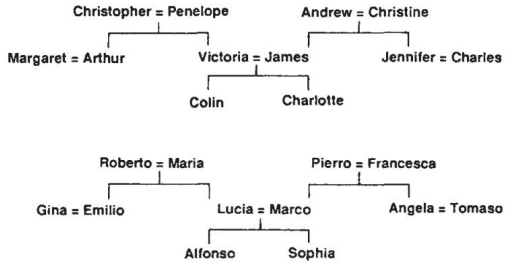



**Fig. 2** Two isomorphic family trees. The information can be expressed as a set of triples of the form ⟨person 1⟩⟨relationship⟩⟨person 2⟩, where the possible relationships are {father, mother, husband, wife, son, daughter, uncle, aunt, brother, sister, nephew, niece}. A layered net can be said to 'know' these triples if it can produce the third term of each triple when given the first two. The first two terms are encoded by activating two of the input units, and the network must then complete the proposition by activating the output unit that represents the third term.

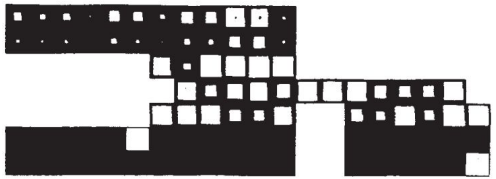


**Fig. 3** Activity levels in a five-layer network after it has learned. The bottom layer has 24 input units on the left for representing ⟨person 1⟩ and 12 input units on the right for representing the relationship. The white squares inside these two groups show the activity levels of the units. There is one active unit in the first group representing Colin and one in the second group representing the relationship 'has-aunt'. Each of the two input groups is totally connected to its own group of 6 units in the second layer. These groups learn to encode people and relationships as distributed patterns of activity. The second layer is totally connected to the central layer of 12 units, and these are connected to the penultimate layer of 6 units. The activity in the penultimate layer must activate the correct output units, each of which stands for a particular ⟨person 2⟩. In this case, there are two correct answers (marked by black dots) because Colin has two aunts. Both the input units and the output units are laid out spatially with the English people in one row and the isomorphic Italians immediately below.

Rhumelhart et al (1986)

# Information bottleneck ‘theory’

- A bit more inside-baseball: Information bottleneck principle (Tishby et al. 1999) and the debate if that’s what’s up with deep nets (Shwartz-Ziv et al, 2017) or not (Saxe et al 2018)

We define the relevant information in a signal $x \in X$ as being the information that this signal provides about another signal $y \in Y$. Examples include the information that face images provide about the names of the people portrayed, or the information that speech sounds provide about the words spoken. Understanding the signal $x$ requires more than just predicting $y$, it also requires specifying which features of $X$ play a role in the prediction. We formalize this problem as that of finding a short code for $X$ that preserves the maximum information about $Y$. That is, we squeeze the information that $X$ provides about $Y$ through a ‘bottleneck’ formed by a limited set of codewords $\tilde{X}$. This constrained optimization problem can be seen as a generalization of rate distortion theory in which the distortion measure $d(x, \tilde{x})$ emerges from the joint statistics of $X$ and $Y$. This approach yields an exact set of self consistent equations for the coding rules $X \to \tilde{X}$ and $\tilde{X} \to Y$. Solutions to these equations can be found by a convergent re–estimation method that generalizes the Blahut–Arimoto algorithm. Our variational principle provides a surprisingly rich framework for discussing a variety of problems in signal processing and learning, as will be described in detail elsewhere.